



KANN AUDITS

Security Review **For N4A**

Prepared For: **N4A**

Date Reported: **August 18, 2026**

Audit Summary

N4A engaged Kann Audits to perform a security review of its on-chain bounty, report adjudication, escrow, and payout system. From May 21 to June 2, 2026, a team of security researchers performed a manual review of the in-scope contracts and additionally triaged findings generated by the Kann AI Security Model. All findings identified during the assessment are documented in this report.

Protocol Overview

N4A is an on-chain security bounty and report adjudication system that manages bounty programs, escrowed funds, researcher reports, judge-based reviews, disputes, and payouts. Reports are tracked through a defined lifecycle with primary review, escalation, second-opinion, time-lock, and final payout flows. The system also manages bounty and judge balances, company penalty debt, program refunds, and access-controlled administrative roles.

Security Researchers

derastephh

AlexScherbatyuk

anchabadze

Volodymyr

Luc1jan

Fishy

heeze

Riceee

Lookman

yusufthebdev

VCeb

DeveloperX

anchabadze

Digital Forensic

Count - sum

Jerry

parvanj

Razzky

Rare_one

Scope

Files:

- `src/BountyProgramRegistry.sol`
- `src/EscrowVault.sol`
- `src/Faucet.sol`
- `src/JudgeRegistry.sol`
- `src/PayoutController.sol`
- `src/ReportManager.sol`
- `src/UsdcFaucet.sol`
- `src/utils/ReentrancyGuard.sol`

Methodology

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms, and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

The auditing process specifically considered:

- Testing against common and uncommon attack vectors
- Ensuring compliance with industry best practices and known smart contract security standards
- Verifying contract logic matches the client's intended behavior
- Cross-referencing patterns with industry-standard implementations
- Manual line-by-line review by experienced auditors
- Access control verification
- Architecture and design analysis to ensure secure component interactions and clearly defined trust boundaries
- State transition and logic review to detect inconsistencies, unexpected behavior, or potential exploitation paths

Risk Classification

- **High** – result in directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
- **Medium** – result in moderate loss of funds or impact multiple users, but only under specific conditions. They are exploitable without privileged access but typically require certain assumptions or edge cases.
- **Low/Info** – issues are non-exploitable findings that do not pose a direct security risk or impact the system's integrity. These issues are typically cosmetic, best-practice deviations, or related to documentation and compliance. While they do not require urgent remediation, they may be addressed to improve code quality and maintainability.

Findings Count

Critical/High	Medium	Low/Info
1	3	6

Findings Summary

ID	Title	Severity
H-01	Unbounded programPenaltyReports Loop Causes Gas-Exhaustion DoS on Deposit, Debt Payment, and Refund	High
M-01	Rejected reports are excluded from blocking report accounting before their dispute window has expired	Medium
M-02	Program can be activated while penalty debt remains unpaid due to proportional rounding during penalty repayment	Medium
M-03	Emergency Close Is Unreachable For Permanently Blocked Researcher Payouts	Medium
L-01	Missing Payout Schedule Requirement in approvePrimary	Low
L-02	Second Opinion Flow Restricts Secondary Judge to Confirm/Downgrade/Invalidate Only, Preventing Honest Severity Upgrades, Compounded by Mutual Exclusivity With Escalation	Low
L-03	withdraw() Always Emits Zero as Withdrawn Amount	Low
I-01	Companies Cannot Reliably Close Active Programs Because Refund Initiation Requires Zero Blocking Reports	Info
I-02	Blacklisted Judge Can Brick Company Penalty Settlement And Block Refunds	Info
I-03	Untracked Pending Obligations Allow bountyBalance Overcommitment Across Concurrent Reports	Info

Detailed Findings

H-01 - Unbounded programPenaltyReports Loop Causes Gas-Exhaustion DoS on Deposit, Debt Payment, and Refund

High

Location

EscrowVault.sol:L368, EscrowVault.sol:L442-L497

Description

Every call to `EscrowVault accrueCompanyPenalty()` at L368 appends a new `bytes32` entry to `programPenaltyReports[programId]`:

```
programPenaltyReports[programId].push(reportId);
```

There is no cap on this array. Entries are never removed. The array grows monotonically with every second-opinion resolve that triggers a company penalty, across all reports associated with a program over its entire lifetime.

`_applyPenaltyPayment()` at L452 iterates the entire array on every invocation:

```
for (uint256 i = 0; i < reportCount; i++) {
    bytes32 rId = reportIds[i];
    uint256 reportDebt = reportPenaltyDebt[rId];
    if (reportDebt == 0) continue;
    // ... transfer logic ...
}
```

The `continue` on fully-paid entries (L455) saves the transfer cost, but the SLOAD for `reportPenaltyDebt[rId]` is still executed for every entry. Paid-off entries therefore remain in the iteration permanently, keeping the loop cost elevated even after all debt is cleared.

`_applyPenaltyPayment()` is called on every `deposit()` (L157), `payPenaltyDebt()` (L222), and `refund()` (L320) when `companyPenaltyDebt[programId] > 0`. This means any program that has accumulated enough penalty accrual events will have these three entry points permanently bricked — all transactions will run out of gas.

Impact

At approximately 9,504-9,591 gas per array entry (measured at $n=200$ and $n=250$), the 30M Ethereum block gas limit is reached at approximately

3,127 cumulative penalty entries. Once this threshold is crossed, `deposit()`, `payPenaltyDebt()`, and `refund()` for the affected program all revert with out-of-gas. The company cannot top up its bounty pool, cannot clear its debt, and cannot recover its escrowed funds. All researchers with open approved reports cannot be paid. There is no admin escape hatch targeting the loop itself. This finding also creates a vector for a compound attack when combined with `emergencyBountyPayout` — see H-06.

Recommendation

Replace the always-on iteration with a paginated or pruning approach. The simplest fix is to remove paid-off entries by swapping them with the last element and popping the array when `reportPenaltyDebt[rId]` is zeroed out during payment processing:

```
// After zeroing reportPenaltyDebt[rId] inside the loop:
// Swap-and-pop to prune the array
reportIds[i] = reportIds[reportCount - 1];
reportIds.pop();
reportCount--;
i--; // re-check the swapped entry
```

Alternatively, introduce a separate `paidReportIds` mapping and a maximum-entries-per-call parameter to bound gas regardless of array length. The swap-and-pop approach is preferred as it eliminates the root accumulation issue without requiring callers to manage pagination.

Remediation

FIXED

M-01 - Rejected reports are excluded from blocking report accounting before their dispute window has expired

Medium

Description and Impact

In `ReportManager.markApprovedPrimary()`, a primary decision with `severity == 0` immediately transitions the report from `SUBMITTED` to `REJECTED`:

```
if (severity == 0) {  
    _transitionStatus(report, Status.REJECTED);  
}
```

However, `REJECTED` is not treated as a blocking state in `ReportManager._isBlockingStatus()`. As a result, moving from `SUBMITTED` to `REJECTED` decrements `blockingReportCount` immediately, even though the rejection is not economically or procedurally final until the report's `timelockEnd` has passed.

The root cause is a mismatch between lifecycle rights and refund-blocking accounting. The protocol still gives the researcher a live post-rejection remedy during the timelock window: `PayoutController.escalateReport()` explicitly allows escalation from `ReportManager.Status.REJECTED` so long as `block.timestamp < report.timelockEnd`. In other words, the report remains contestable, but the blocking-report accounting treats it as fully resolved.

This inconsistency allows the registry to believe there are no open blocking reports when, in reality, a rejected report can still be escalated. Since `BountyProgramRegistry.initiateRefund()` only checks `hasBlockingReports(programId)`, a company may be able to initiate the refund flow while a researcher's escalation window is still open. Once that happens, the program can transition toward `PAUSED` and ultimately `CLOSED` despite the fact that the report was not yet final from the researcher's perspective.

The impact is that researchers can lose their practical ability to challenge a primary rejection, because the protocol may permit shutdown and refund actions during the active dispute window. This creates a race between the company's refund initiation and the researcher's escalation right. Operationally, it weakens the dispute-resolution guarantees of the system and can cause reports to be treated as finalized too early, resulting in premature fund withdrawal, broken report lifecycle assumptions, and unfair loss of recourse for researchers.

Recommendation

The best mitigation is to keep rejected reports in the blocking set until their dispute window has actually expired, or until they become truly final through some explicit terminal transition.

- Treat REJECTED as a blocking status in `_isBlockingStatus()` for the duration of the escalation window.

Remediation

FIXED

M-02 - Program can be activated while penalty debt remains unpaid due to proportional rounding during penalty repayment

Medium

Root Cause

`_applyPenaltyPayment()` distributes the repayment proportionally across all penalized reports using integer division:

```
uint256 reportPayment = (penaltyPaid * reportDebt) / totalDebt;
```

Because integer division rounds down, some reports may receive `reportPayment == 0` and are skipped entirely. Consequently, only `actualPaid` is applied toward reducing `companyPenaltyDebt`, while the remaining deposited funds are treated as a normal program deposit.

Description

When a program owner deposits funds, the protocol first attempts to repay any outstanding penalty debt by calling `_applyPenaltyPayment()`. However, proportional allocation using integer division can round some report allocations down to zero, causing those reports to receive no repayment.

Only the successfully allocated amount (`actualPaid`) is deducted from the deposit, while the remaining funds continue through the normal deposit flow, increasing the program's bounty balance. At the end of `deposit()`, the protocol activates the program even though some penalty reports may still have outstanding debt because they were skipped during the proportional distribution.

Impact

A program may transition to the ACTIVE state before all penalty obligations have been settled.

As a result:

`treasuryPay`, `primaryPay`, and `secondaryPay` are never executed for those reports, part of the deposit intended to repay penalties is instead used to fund the program's bounty pool, the protocol reaches an inconsistent state where the program is active despite outstanding penalty debt.

Remediation

FIXED

M-03 - Emergency Close Is Unreachable For Permanently Blocked Researcher Payouts

Medium

Description

`emergencyCloseReport()` is intended to close reports whose normal bounty payout cannot be completed, such as when the researcher is blacklisted by USDC. That recovery path is unreachable because it only accepts reports in `READY_TO_PAY`, but no execution path leaves a report in that state across transactions.

In `finalizeAndPay()`, reports coming from `APPROVED_PRIMARY`, `ESCALATED_RESOLVED`, or `SECOND_OPINION_RESOLVED` are first transitioned to `READY_TO_PAY` through `reports.markReadyToPay(reportId)`. The same transaction then immediately continues to `reports.markPaid(reportId)` and `escrow.payoutBounty(programId, researcher, payoutAmount)`.

If the bounty transfer fails because the payout token blocks transfers to the researcher, `EscrowVault.payoutBounty()` reverts with `TRANSFER_FAILED`. That revert rolls back the entire `finalizeAndPay()` call, including the earlier `markReadyToPay()`. As a result, the report never persists in `READY_TO_PAY`, so the admin-only `emergencyCloseReport()` function can never be used for the exact stuck-payout scenario it was added to handle.

Impact

The documented emergency recovery flow for blacklisted researchers is non-functional. When a normal payout is permanently blocked, the report remains stuck in a blocking adjudicated status such as `APPROVED_PRIMARY`, `ESCALATED_RESOLVED`, or `SECOND_OPINION_RESOLVED`, and the admin cannot close it through `emergencyCloseReport()`.

Root Cause

`src/PayoutController.sol` assumes `READY_TO_PAY` can act as a durable intermediate state for emergency handling, but `finalizeAndPay()` both enters and exits that state in the same transaction.

Vulnerability Path

1. A report reaches `APPROVED_PRIMARY`, `ESCALATED_RESOLVED`, or

SECOND_OPINION_RESOLVED and becomes eligible for final payout.

2. The assigned judge calls `finalizeAndPay()`.

3. `finalizeAndPay()` calls `reports.markReadyToPay(reportId)` and then proceeds toward `reports.markPaid(reportId)` and `escrow.payoutBounty(programId, researcher, payoutAmount)`.

4. The payout token rejects the transfer to the researcher, such as because the researcher address is USDC-blacklisted.

5. `EscrowVault.payoutBounty()` reverts, which reverts the full `finalizeAndPay()` transaction and undoes the earlier `READY_TO_PAY` transition.

6. The report remains in its pre-existing adjudicated status instead of persisting in `READY_TO_PAY`.

7. The admin calls `emergencyCloseReport(reportId)`, but it reverts with `NOT_READY_TO_PAY`, so the intended emergency close path is unavailable.

Remediation

FIXED

L-01 - Missing Payout Schedule Requirement in approvePrimary

Low

Problem

The protocol allows the creation of "empty" programs (no payout schedule), but the approval logic expects one. This creates a "dead-end" workflow.

Logic Breakdown

Creation: `_storePayoutSchedule` allows `severityCount == 0`, resulting in `hasPayoutSchedule = false`. Validation: `approvePrimary` contains: `require(registry.hasPayoutSchedule(programId), "SCHEDULE_REQUIRED");`.

Impact

A company can successfully launch a program and researchers can submit work. However, when a judge attempts to approve a report, the transaction will always revert. The program is essentially a trap where work is performed but cannot be processed through the standard primary approval path.

Recommendation

Enforce that a payout schedule with at least one severity must be provided during the program initialization phase in the registry.

Remediation

FIXED

L-02 - Second Opinion Flow Restricts Secondary Judge to Confirm/Downgrade/Invalidate Only, Preventing Honest Severity Upgrades, Compounded by Mutual Exclusivity With Escalation

Low

Description

The `PayoutController::approveSecondOpinion()` function constrains the secondary judge to three outcomes: confirm the primary severity (`outcome == 0`), downgrade it (`outcome == 1`), or invalidate the finding entirely (`outcome == 2`). There is no outcome that allows the secondary judge to upgrade the severity above what the primary judge assigned.

```
// outcome 0 -- CONFIRM: severity must exactly match primary
if (outcome == 0) {
    require(severity == report.primarySeverity, "SEVERITY_MISMATCH");
}
// outcome 1 -- DOWNGRADE: severity must be strictly less than primary
else if (outcome == 1) {
    require(severity > 0, "INVALID_SEVERITY");
    // ...
    require(severity < report.primarySeverity, "SEVERITY_NOT_DOWNGRADE");
}
// outcome 2 -- INVALIDATE: severity must be 0
else if (outcome == 2) {
    require(severity == 0, "INVALIDATE_SEVERITY");
}
```

If the secondary judge genuinely believes the finding is a higher severity than the primary judge assessed, their only option is to confirm at the primary severity — they cannot express their honest judgment. This is particularly damaging when combined with the fact that escalation and second opinion are mutually exclusive: `PayoutController::requestSecondOpinion()` requires `APPROVED_PRIMARY` status, and `PayoutController::escalateReport()` transitions the report to `ESCALATED`. Whichever party acts first during the timelock window locks out the other. If the company front-runs the researcher's escalation by requesting a second opinion, the researcher permanently loses their ability to dispute upward, and the second judge is simultaneously prevented from correcting the severity upward.

Note: This is also not a particularly fair system, as when a researcher escalates a report the second judge is free to confirm, upgrade or downgrade the severity. But as mentioned in the case of a second opinion request by a team, the severity can only be confirmed or reduced by the second judge.

Attack Path

1. A researcher submits a report. The primary judge approves it at severity 2 (Medium). The report enters `APPROVED_PRIMARY` with a timelock window.
2. The researcher believes the finding is Critical (severity 4) and intends to escalate.
3. The company calls `requestSecondOpinion()` first — the report status moves to `SECOND_OPINION_REQUESTED`.
4. The researcher can no longer call `escalateReport()` because the report is no longer in `APPROVED_PRIMARY` or `REJECTED` status.
5. The admin assigns a secondary judge. The secondary judge reviews the report and believes the finding is Critical.
6. The secondary judge cannot set severity 4. Their only options are: confirm Medium (outcome 0), downgrade to Low (outcome 1), or invalidate (outcome 2).
7. The secondary judge is forced to confirm at Medium — the researcher receives a Medium payout for a Critical finding.

Root Cause

`PayoutController::approveSecondOpinion()` only defines three outcome paths (confirm, downgrade, invalidate) with no upgrade path. The `PayoutController::requestSecondOpinion()` function requires `Status.APPROVED_PRIMARY`, while `PayoutController::escalateReport()` also requires `APPROVED_PRIMARY` or `REJECTED` — making them mutually exclusive once either is called. The `ReportManager::markSecondOpinionRequested()` does accept `ESCALATED` status, suggesting the original design may have intended to allow both flows, but `PayoutController::requestSecondOpinion()` does not use that path.

Impact

- The secondary judge cannot give an honest assessment when they believe severity should be higher, undermining the integrity of the dispute resolution system.
- A company can strategically front-run a researcher's escalation by requesting a second opinion first, knowing the outcome can only stay the same or go lower.
- Researchers can be permanently underpaid for legitimate high-severity findings with no remaining recourse — they cannot escalate, and the second judge cannot upgrade on their behalf.

Recommendation

Add an UPGRADE outcome (`outcome == 3`) to `approveSecondOpinion()` that allows the secondary judge to set a severity higher than the primary:

```
function approveSecondOpinion(
    uint256 programId,
    address researcher,
    bytes32 reportHash,
    uint8 severity,
    uint8 outcome
) external nonReentrant {
    require(judges.isJudge(msg.sender), "NOT_JUDGE");
    require(researcher != address(0), "RESEARCHER_ZERO");
    require(reportHash != bytes32(0), "HASH_ZERO");
-   require(outcome == 0 || outcome == 1 || outcome == 2, "INVALID_OUTCOME");
+   require(outcome <= 3, "INVALID_OUTCOME");

    // ... existing config/status checks ...

    if (outcome == 0) {
        require(severity == report.primarySeverity, "SEVERITY_MISMATCH");
        payoutAmount = report.payoutAmount;
        companyPenaltyBps = config.companyPenaltyBps;
    } else if (outcome == 1) {
        // ... existing downgrade logic ...
    } else if (outcome == 2) {
        require(severity == 0, "INVALIDATE_SEVERITY");
        judgePenaltyBps = config.judgePenaltyInvalidBps;
        payoutAmount = 0;
+   } else if (outcome == 3) {
+       require(severity > report.primarySeverity, "SEVERITY_NOT_UPGRADE");
+       require(severity < 5, "INVALID_SEVERITY");
+       payoutAmount = registry.payoutBySeverity(programId, severity);
+       require(payoutAmount > 0, "SEVERITY_NOT_CONFIGURED");
+       judgePenaltyBps = config.judgePenaltyDowngradeBps;
    }
}
```

Additionally, consider allowing `escalateReport()` to work on reports in `SECOND_OPINION_REQUESTED` status, so that both parties can express their disagreement with the primary judgement independently.

Remediation

FIXED

L-03 - withdraw() Always Emits Zero as Withdrawn Amount

Low

Description

The `withdraw` function sends the entire contract balance to the admin and then emits `Withdrawn` with `address(this).balance` as the amount. Because the ETH transfer executes before the event emission, `address(this).balance` is already zero at the point the event is emitted. Every `Withdrawn` event will log an amount of zero regardless of how much ETH was actually withdrawn.

Root Cause

The balance is read after the transfer rather than before it:

```
function withdraw() external onlyAdmin {
    (bool ok,) = payable(msg.sender).call{value: address(this).balance}("");
    require(ok, "ETH_TRANSFER_FAILED");
    emit Withdrawn(msg.sender, address(this).balance); // balance is now 0
}
```

Impact

Off-chain systems, monitoring dashboards, and audit trails that rely on events to track fund flows will always see a withdrawn amount of zero. This makes the event log unreliable for accounting purposes.

Recommendation

Capture the balance before the transfer and use the cached value in the event:

```
function withdraw() external onlyAdmin {
    uint256 amount = address(this).balance;
    (bool ok,) = payable(msg.sender).call{value: amount}("");
    require(ok, "ETH_TRANSFER_FAILED");
    emit Withdrawn(msg.sender, amount);
}
```

Remediation

FIXED

I-01 - Companies Cannot Reliably Close Active Programs Because Refund Initiation Requires Zero Blocking Reports

Info

Description

`initiateRefund()` can only succeed when the target program has zero blocking reports, but the program is not paused until after that check passes. While the program remains `ACTIVE`, new reports can still be submitted, and each new `SUBMITTED` report immediately becomes blocking.

As a result, a company cannot use the refund flow to first stop report intake and then resolve any remaining reports. Instead, the company must wait for the blocking report count to naturally reach zero while the program is still live. For a bounty program with ongoing report activity, that state may never occur in practice.

This makes closure dependent on the program already being effectively idle. A company that wants to shut down an active bounty has no protocol-controlled way to stop new reports from arriving before satisfying the zero-blocking-report condition, so the refund flow can be denied indefinitely by ordinary report inflow.

Remediation

FIXED

I-02 - Blacklisted Judge Can Brick Company Penalty Settlement And Block Refunds

Info

Description

When a company requests second opinion and the second judge confirms the original approval (`outcome == 0`), the protocol charges the company a penalty. `approveSecondOpinion()` sets `companyPenaltyBps` for that case and calls `escrow accrueCompanyPenalty(...)`, which records the penalty debt together with the primary and secondary judge addresses that must receive the judges' shares of the payment.

Later, every path that clears company penalty debt routes through `EscrowVault._applyPenaltyPayment(): payPenaltyDebt(), refund()`, and even `deposit()` when an outstanding penalty exists. `_applyPenaltyPayment()` attempts to transfer the treasury share and both judge shares with strict `require(token.transfer(...))` checks.

If either stored judge address is blacklisted by the payout token (USDC), the corresponding transfer reverts and the entire debt-clearing transaction reverts. Because the penalty debt remains outstanding and every settlement path reuses the same helper, the company cannot clear the debt through manual payment and `executeRefund()` cannot complete either.

Impact

A single blacklisted judge address inside a confirmed second-opinion penalty can permanently lock the program's penalty debt. The company cannot settle the debt through `payPenaltyDebt()`, and once the program is paused and ready for closure, `executeRefund()` also reverts while trying to distribute that judge's penalty share.

Root Cause

`src/PayoutController.sol` accrues company penalty debt on confirmed second-opinion outcomes:

```
if (companyPenaltyBps > 0 && payoutAmount > 0) {
    uint256 penaltyAmount = (payoutAmount * companyPenaltyBps) / 10_000;
    if (penaltyAmount > 0) {
        escrow accrueCompanyPenalty(programId, reportId, penaltyAmount, report.
            primaryJudge, report.secondaryJudge);
    }
}
```

`src/EscrowVault.sol` then stores the judges as mandatory payout recipients and later enforces direct token transfers to them inside `_applyPenaltyPayment()`:

```
require(token.transfer(pj, primaryPay), "PENALTY_PRIMARY_TRANSFER_FAILED");
require(token.transfer(sj, secondaryPay), "PENALTY_SECONDARY_TRANSFER_FAILED");
```

Because `payPenaltyDebt()`, `refund()`, and `deposit()` all call `_applyPenaltyPayment()`, any failed transfer to one blacklisted judge reverts the full penalty-settlement operation and leaves `companyPenaltyDebt` uncleared.

Vulnerability Path

1. A company requests second opinion for an approved report.
2. The assigned second judge returns `outcome == 0` and confirms the primary decision.
3. `approveSecondOpinion()` computes the company penalty and calls `accrueCompanyPenalty(programId, reportId, penaltyAmount, report.primaryJudge, report.secondaryJudge)`.
4. The penalty debt is stored along with the two judge addresses that must receive the judges' shares when the debt is later settled.
5. One of those judge addresses is blacklisted by the payout token, such as USDC.
6. The company later tries to clear the debt through `payPenaltyDebt()` or reaches `executeRefund()`, which calls `EscrowVault.refund()`.
7. `_applyPenaltyPayment()` attempts `token.transfer(...)` to the blacklisted judge, reverts, and rolls back the full transaction.
8. The company penalty debt remains outstanding, and the company cannot complete the refund flow for that program.

Remediation

ACKNOWLEDGED

I-03 - Untracked Pending Obligations Allow bountyBalance Overcommitment Across Concurrent Reports

Info

Summary

Solvency checks in `approvePrimary` and `finalizeEscalation` validate `bountyBalance` against each report's payout in isolation, without accounting for funds already committed to other approved but unpaid reports. This allows the same funds to be implicitly promised to multiple reports simultaneously, with the shortfall only surfacing when `finalizeAndPay` is called.

Root Cause

The contract lacks a `pendingObligations` tracker per `programId`. Every solvency check measures raw `bountyBalance` instead of uncommitted balance, meaning concurrent approved reports can collectively exceed available funds.

Impact

Any report whose committed funds are later consumed by another finalization becomes permanently unpayable — `finalizeAndPay` reverts indefinitely, `blockingReportCount` is never decremented, and the program refund mechanism is blocked.

Recommendation

Introduce a per-program `pendingObligations[programId]` that will track how much money is already committed for a program.

Remediation

ACKNOWLEDGED

Disclaimers

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource, and expertise-bound effort in which we aim to identify as many issues as possible.

About Kann Audits

Kann Audits is a top-tier Web3 security audit company, trusted by leading projects and providing comprehensive audits and expert guidance to secure the most critical blockchain protocols.

Kann Audits provides services such as manual auditing, AI audits using the Kann AI Labs tool, and incident response.

To learn more, visit <https://kannaudits.com>

To view our audit portfolio, visit <https://github.com/KannAudits>

To book an audit, message <https://t.me/KannAudits>